

Java

Zuletzt aktualisiert am:

27.09.2026

Inhaltsverzeichnis

1. Die <code>for</code> -Schleife	1
2. Datenlisten	2
2.1. Arrays	2
2.2. Referenztypen	5

1. Die for-Schleife

Folgende Schleife:

java

```
int i = 0;
while (i < 10)
{
    // ...
    i++;
}
```

... kann auch so geschrieben werden:

java

```
for (int i = 0; i < 10; i++)
{
    // ...
}
```

Drei Ausdrücke werden in den Schleifenkopf geschrieben: `for (A; B; C):`

A: Start Dieser Ausdruck wird **nur einmal** vor Beginn des ersten Durchgangs ausgeführt. Meistens wird hier eine Zählvariable initialisiert.

B: Bedingung Hier steht der boolesche Ausdruck für die Abbruchbedingung, wie Sie ihn schon aus der `while`-Schleife kennen: Sobald die Bedingung `false` ist, wird die Schleife beendet.

C: Update Dieser Ausdruck wird immer am Ende eines Durchganges ausgeführt. Hier wird normalerweise die Zählvariable (aus A) hoch/runter gezählt. Diesen Ausdruck könnten Sie auch als letzten Befehl in den Schleifenkörper schreiben. Er steht aber oben mit im Schleifenkopf für bessere Lesbarkeit.

Die `for`-Schleife wird fast immer wie im oberen Beispiel verwendet. Sie können aber beliebige Ausdrücke verwenden, solange die Syntax es erlaubt:

java

```
for (double piePiece = 1.0; piePiece > 0.1; piePiece = piePiece / 2.0)
{
    IO.println("Kuchenstück: " + piePiece);
}
```

Alle drei Ausdrücke sind optional und können auch leer gelassen werden. In diesem Beispiel wurden sogar alle drei Ausdrücke weggelassen:

java

```
// Endlosschleife mit FOR
for(;;) // <-- Alle drei Ausdrücke wurden weggelassen
{
    IO.println("Ist Mittagessen fertig?");
}

// Endlosschleife mit WHILE
while(true)
{
    IO.println("Ist Mittagessen fertig?");
}
```

2. Datenlisten

Eine Variable ist wie eine Schachtel, die einen Wert mit einem bestimmten Datentypen enthält. Der Name der Variable (der *Bezeichner*) ist das Label auf der Schachtel und der Datentyp bestimmt die Größe der Schachtel.

java

```
String article = "Mehl";
```

Alle unsere Schachteln liegen im Hauptspeicher des Systems und unser Programm findet die Werte mit denen wir arbeiten wollen anhand der Labels auf den Schachteln wieder.

Wie arbeiten wir aber mit Auflistungen von Werten? Wenn wir eine Einkaufsliste mit vier unterschiedlichen Artikeln darstellen wollen, dann könnten wir vier String-Variablen deklarieren und nummerieren:

java

```
String article_1 = "Mehl";
String article_2 = "Salz";
String article_3 = "Hot Wheels® Looping Set";
String article_4 = "Hefe";
```

Für jeden Artikel eine Variable anzulegen wird schnell sehr unhandlich. Versuchen Sie mal den User nach einer Zahl zu fragen und anschließend den entsprechenden Artikel anhand der Position in der Einkaufsliste in der Konsole auszugeben (wir ignorieren Mal fehlerhafte Eingaben).

java

```
String num_str = IO.readLine("Wählen Sie einen von vier Artikeln aus (1 bis 4): ");
int num        = Integer.parseInt(num_str);

if (num == 1) { IO.println(article_1); }
if (num == 2) { IO.println(article_2); }
if (num == 3) { IO.println(article_3); }
if (num == 4) { IO.println(article_4); }
```

Eine Schleife kann uns an dieser Stelle auch nicht weiterhelfen. Dazu kommt, dass unsere Lösung nicht mit hunderten Artikeln gut skaliert.

Und was ist, wenn wir den User neue Artikel hinzufügen lassen möchten? Wir können in Java keine neuen Variablen *zur Laufzeit* der Software deklarieren. Alle unsere Variablen müssen (in Java) vor Auslieferung der Software im Quellcode schon deklariert sein.

Aufbauend auf unseren primitiven Datentypen (und `String`) lernen wir jetzt unsere erste sog. **Datenstruktur** kennen.

2.1. Arrays

Die simpelste Datenstruktur ist das **Array** und existiert in den meisten Programmiersprachen. Wir können eine Liste an Werten in einem Array speichern, wenn:

1. alle Werte den **gleichen Datentypen** haben und
2. wir **vorher** wissen **wie viele** Elemente das Array hat.

Stellen Sie sich ein Array vor wie ein Regal. Sie bauen (deklarieren) ein Regal mit `x` Fächern. Jedes Fach hat die gleiche Größe und die Anzahl der Fächer im Regal ändert sich nicht.

```
String[] myShelf = new String[5];
```

Anstatt neue Variablen für jeden Wert anzulegen, können wir in unserem Array unsere Werte in der »Regalfachnummer« ablegen.

Aber Vorsicht: Die Fachnummer (der sog. **Index**) beginnt bei 0!



```
myShelf[0] = "Putzmittel";
```

```
myShelf[1] = "Drucker";
```

```
myShelf[2] = "alter Laptop";
```

```
myShelf[3] = "Akkubohrer";
```

```
myShelf[4] = "Kabelkiste";
```

java

```
String[] myShelf = new String[5]; // Deklaration
myShelf[0] = "Putzmittel";         // Initialisierung
myShelf[1] = "Drucker";           // Initialisierung
myShelf[2] = "alter Laptop";       // Initialisierung
myShelf[3] = "Akkubohrer";        // Initialisierung
myShelf[4] = "Kabelkiste";        // Initialisierung

String index_str = IO.readLine("Wählen Sie einen von fünf Fächern aus (1 bis 5): ");
int index        = Integer.parseInt(index_str);

// Index um 1 reduzieren, weil wir Informatiker, anders als der User,
// bei 0 anfangen zu zählen.
index--;
IO.println(myShelf[index]);
```

In diesem Array können wir fünf Strings speichern und auch *nur Strings* und *nur genau fünf* davon.

Ein Array ist nur eine Datenstruktur. In Kombination mit einem *inneren* Datentyp wird ein neuer Datentyp daraus.. Hier sind vier Beispiele aufgelistet:

java

```
int[]    numArr    = new int[1000];
byte[]   streamBuffer = new byte[1024];
boolean[] myBools   = new boolean[4];
String[] weekDays   = new String[7];
```

Tatsächlich können Sie mit allen Datentypen, die Sie in Zukunft kennenlernen werden, eine Array-Variante deklarieren, indem Sie `[]` hinter den Datentypen schreiben.

Schlüsselwort: `new`

Arrays sind unsere nächsten komplexen Datentypen, neben String, die wir kennenlernen. Genau wie Strings sind Arrays Objekte. Objekte werden mit dem `new`-Schlüsselwort *initialisiert*:

```
String[] myShelf;           // Deklaration
myShelf = new String[5];    // Initialisierung
```

java

Bei der Initialisierung von Strings reichen die doppelten Anführungsstriche. Die Syntax mit dem `new`-Schlüsselwort funktioniert aber auch:

```
String name1 = "Herbert";
String name2 = new String("Herbert");
```

java

Dieses Schlüsselwort gehört zur *Objektorientierten Programmierung* und wird dort weiter vertieft.

Standardinitialisierung

Interessant ist folgendes Beispiel:

```
int[] myNumberArray = new int[5];
IO.println(myNumberArray[3]);
```

java

Funktioniert das? Was wird hier ausgegeben?

Kurzschreibweise

Auch ein Array kann mit nur einem Befehl deklariert und initialisiert werden:

```
double[] myGrades = {6.0, 6.0, 6.0, 5.9, 6.0};
IO.println(myGrades[3]); // Ausgabe: 5.9
```

java

Arrays sind eine sehr simple aber auch unflexible Datenstruktur. Sobald die Größe festgelegt ist, können keine Elemente mehr hinzugefügt werden und wir können auch keine Datentypen mischen. Neben dem Array werden wir in Zukunft noch einige andere Datenstrukturen kennenlernen und sogar eigene programmieren. Diese Datenstrukturen arbeiten aber sehr häufig im Hintergrund mit Arrays.

2.2. Referenztypen

Arrays sind auch unser erster *Referenztyp* den wir kennenlernen. Bei unseren primitiven Datentypen wird ein Wert von einer Variable mit dem Zuweisungsoperator in die andere kopiert:

java

```
double temperature    = 19.3;
double temperatureNew = temperature;

temperatureNew += 0.5;

IO.println("Die Temperatur ist von " + temperature
           + " auf " + temperatureNew + " gestiegen.");
```

Betrachten Sie im Vergleich dazu folgendes Beispiel. Testen Sie es in Ihrer IDE aus:

java

```
// Erstes Regal definieren
String[] myFirstShelf = new String[3];
myFirstShelf[0] = "Putzmittel";
myFirstShelf[1] = "Drucker";
myFirstShelf[2] = "alter Laptop";

// Regal "kopieren"
String[] mySecondShelf = myFirstShelf;
mySecondShelf[2] = "neuer Laptop"; // Ein Element im neuen Regal ändern

// Drittes Fach von beiden Regalen ausgeben
IO.println("Originales Regal (Fach 3): " + myFirstShelf[2]);
IO.println("Kopiertes Regal (Fach 3): " + mySecondShelf[2]);
```

Wenn Sie Ihre Ausgabe betrachten und weiter testen, werden Sie feststellen, dass immer noch nur *ein* Array im Speicher existiert. Nur haben Sie jetzt zwei Variablen, die das gleiche Array **referenzieren**. Diese Eigenschaft wird Ihnen beim Programmieren noch sehr oft begegnen. Komplexe Datentypen sind fast immer **Referenztypen**.

So *klonen* Sie ein Array übrigens richtig, wenn Sie eine unabhängige Kopie brauchen:

java

```
String[] mySecondShelf = myFirstShelf.clone();
```

Aus genau diesem Grund vermeiden wir den Vergleichsoperator `==`, wenn wir Strings miteinander vergleichen. In dem Fall werden Referenzen und nicht Werte miteinander verglichen. Das gleiche Problem haben Sie auch mit Arrays. Um bei dem Beispiel mit den beiden Regalen zu bleiben, müssen Sie tatsächlich jedes »Regalfach« einzeln mit dem anderen Regal vergleichen. Nur so können Sie den Inhalt zweier Arrays miteinander vergleichen: Element für Element (z. B. in einer Schleife).

Arraylänge



Ähnlich wie mit Strings können Sie auch an die Arraylänge herankommen.

java

```
IO.println(someArray.length); // Keine runden Klammern nach .length
IO.println(someString.length()); // Runde Klammern nicht vergessen!
```

Nützlich um Arrays mit einer Schleife zu durchlaufen!